GPT-SaaS：Agent 构建、数据库演进与学习路线

基于当前代码仓库的架构审查与渐进式实践方案

结论先行

当前仓库已经是一条可运行、可追溯的 AI 知识编译流水线，但还不是完整的自主 Agent。最适合你的下一步，不是引入多 Agent 框架，也不是立刻迁移 PostgreSQL，而是先在独立虚拟环境中读懂现有数据流，再用现有服务封装出一个单 Agent、有限步数、工具分级、全程留痕的最小闭环。

审查对象	结果
代码版本	pyproject.toml 标记 0.3.0；Python >= 3.11
当前架构	Python 标准库模块化单体 + SQLite + CLI/本地 HTTP + 静态 UI
自动验证	2026-09-16 使用 Python 3.12.14 运行：15/15 通过
数据库状态	实际 SQLite Schema 共 20 张表；默认 data/ai-memory.sqlite 当前未创建
本报告范围	只读审查、环境建议、Agent/数据库设计、8 周学习路线；未改业务代码

审查日期：2026-09-16 · 工作目录：D:\mytools\GPT-SaaS
阅读建议：先看第 1、3、6、8 节，再按第 7 节开始动手。

目录

目录 2

1. 审查结论：先把它当作“知识编译器”，再逐步长成 Agent 3

2. 当前代码仓库是怎样工作的 3

2.1 模块地图 3

2.2 当前已经做对的事情 3

2.3 目前的边界 4

3. 你的第一个 Agent 应该怎样设计 4

3.1 最小 Agent 的六个组成部分 4

3.2 第一批工具：先读后写 5

3.3 建议的数据契约 5

4. 数据库审查与建议 5

4.1 现有 20 张表可以按五层理解 5

4.2 SQLite 继续使用的理由与迁移门槛 5

4.3 为 Agent 新增的数据表（将来实现，不是现在改） 5

4.4 数据库具体改进顺序 6

5. 新虚拟环境：建议现在就做 6

5.1 Windows PowerShell 推荐命令 6

5.2 用独立学习数据库跑一遍 6

6. 代码阅读顺序：不要从 UI 开始 7

7. 8 周渐进学习路线 7

7.1 每周固定复盘问题 7

8. 建议的实践项目：从读懂到真正拥有 8

练习 A：数据库侦探（第 1-2 周） 8

练习 B：确定性“小 Agent”（第 3-4 周） 8

练习 C：只读 LLM Agent（第 5 周） 8

练习 D：需要批准的编译 Agent（第 6 周） 8

练习 E：检索评测（第 7 周） 8

9. 实现路线图：未来真正动代码时怎么排 8

9.1 不建议现在做的事 8

10. 风险清单与应对 9

11. 现在就可以执行的 10 项清单 9

附录 A：审查证据与定位 10

附录 B：迁移 PostgreSQL 的决策表 10

附录 C：核心术语速查 10

1. 审查结论：先把它当作“知识编译器”，再逐步长成 Agent

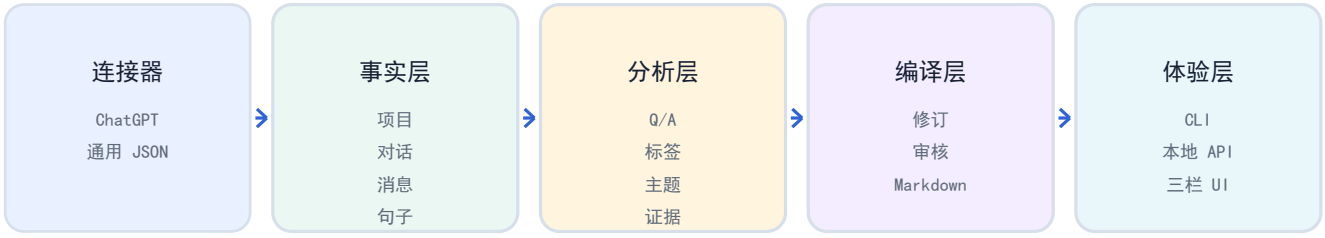
仓库的核心价值不是“模型会不会聊天”，而是把来源、规范化数据、AI 推导、证据、修订和人工审核分开。这是构建可靠 Agent 最难得的地基。

判断	结论	对你的行动含义
现在是不是 Agent？	部分是。已有可替换 Extractor 和分析流水线，但没有通用工具注册、循环决策、运行状态机、预算控制与工具调用账本。	先把现有服务变成工具，再做单 Agent 控制器。
现在要不要 PostgreSQL？	暂时不要。SQLite 足以支持个人本地学习、离线导入和单进程演示。	先掌握关系模型、事务、索引和迁移；满足迁移门槛后再切换。
是否需要新虚拟环境？	需要，而且优先级最高。项目自身零第三方运行依赖，环境隔离成本很低。	用 Python 3.12 创建 .venv，并把学习数据库与真实数据分开。
是否需要 Agent 框架？	第一版不需要。框架会遮住状态、工具、证据与失败处理。	用 Protocol/dataclass/JSON Schema 手写最小循环；稳定后再评估框架。
下一阶段最重要的指标	证据正确率、工具调用成功率、错误合并率、人工驳回率，而不是“回答看起来聪明”。	先建小型金标集和可重复评测。

推荐优先级：P0 独立环境与基线；P1 读懂数据链和 SQL；P2 单 Agent + 只读工具；P3 运行账本、写操作审批与评测；P4 触发条件满足后迁移 PostgreSQL。

2. 当前代码仓库是怎样工作的

主流程不是黑盒。每一层都能在代码里找到明确入口，也能从结果回到原句。



当前主链：导入 -> 可追溯事实 -> Q/A/主题 -> 知识文件 -> 人工审核

2.1 模块地图

模块	当前职责	你应重点观察
ai_memory/connectors/	把 ChatGPT 导出或通用 JSON 转成统一 Conversation/Message。	适配器模式；输入边界；parent_external_id 如何保留问答关系。
service.py	导入、稳定 ID、幂等去重、句子切分、问题与直接回答关联、搜索。	一次导入的事务边界；内容哈希与 UUID5；原始 JSON 的保留。
database.py	SQLite 连接、WAL、外键、建表和极简升级。	20 张表如何分层；哪些是事实、哪些是派生结果。
qa.py	按对话生成 Q/A 单元、缓存、主题标签、相似标签人工复核。	哈希只判断精确相同；相似度只产生 review，不静默合并。
extractors/	离线规则与可选 OpenAI 结构化提取器，共享 Protocol。	输入/输出契约；证据 ID 白名单；外部调用可替换性。
compiler.py	来源指纹、主题合并、证据关系、文档修订、Markdown 物化。	增量跳过、修订不可覆盖、审核状态变化。
api.py + static/	本地 HTTP API、三栏查看与审核界面。	只监听本机；API 层仍直接组合服务，没有鉴权与多租户。
tests/	15 项回归测试覆盖导入、证据、编译、缓存、API、时间线与审核。	把测试当作可执行规格，按版本顺序阅读。

2.2 当前已经做对的事情

- 证据优先：Topic 和 Q/A 都通过 sentence_id 回指原句，LLM 返回未知证据 ID 会被拒绝。
- 派生结果可重算：compiler_runs、source_fingerprint、config_fingerprint 和 analysis_cache 使重复运行可跳过。
- 人工判断有位置：知识文档有 review_status，标签近似只进入 merge_reviews。
- 隐私默认保守：离线规则是默认路径；可选 OpenAI 请求使用 store=false，API Key 不入库。
- 本地闭环完整：CLI、API、静态 UI、知识文件和 Context Package 已经连通。

2.3 目前的边界

边界	影响
没有真正的 Agent 循环	模型只能完成一次抽取；不能按目标选择多个工具、观察结果、重试或结束。
Schema 升级机制很轻	建表脚本加单次 ALTER，随着版本增加会难以回滚、审计和跨数据库迁移。
全文搜索是 LIKE '%term%'	数据增大后普通索引很难发挥作用，中文检索也缺少分词/召回设计。
SQLite 时间字段混用 TEXT/REAL	查询、排序、跨库迁移和统一时区处理会增加认知负担。
编译写数据库与写文件不是同一原子事务	数据库成功后文件写入失败，或相反，都可能需要补偿/重跑。
无鉴权、无 workspace/user 隔离	只能本机个人使用，不能直接暴露公网。
PostgreSQL 草案与 SQLite 实际模型不完全一致	迁移前要先做字段级映射，而不是直接执行 infra/postgres/001_initial.sql。

3. 你的第一个 Agent 应该怎样设计

建议做一个“证据研究 Agent”，目标是回答某个主题在历史对话中有哪些可信结论，并输出可追溯草稿。它只调用当前系统已经存在的能力，不先引入网页浏览、代码执行或多 Agent。



任何写操作都要经过策略检查；每一步写入运行账本，证据与推导分层保存

3.1 最小 Agent 的六个组成部分

组件	最小实现	必须保存/限制的内容
AgentState	一个 dataclass 或 TypedDict。	run_id、目标、项目范围、当前步骤、剩余预算、已用工具、证据 ID、草稿、状态。
Planner	每轮只选择一个下一动作。	允许动作集合；最大 6-8 步；禁止自由生成工具名。
Tool Registry	把现有 service/compiler/experience 查询包装为结构化函数。	参数 Schema、权限等级、超时、结果大小上限、是否有副作用。
Policy Gate	工具执行前检查。	只读自动允许；写操作要求显式模式或人工批准；禁止任意 SQL/文件路径。
Evidence Critic	验证最终陈述是否引用真实 sentence_id。	未知 ID 拒绝；区分历史归纳与 AI 补充；缺证据时返回“无法确认”。

组件	最小实现	必须保存/限制的内容
Run Ledger	追加式记录每一步。	输入摘要、工具、参数、结果摘要、错误、模型/提示版本、token/费用、起止时间。

3.2 第一批工具：先读后写

等级	建议工具	来源/说明
L0 只读	list_projects、list_questions、search_sentences、get_topic、timeline、context_package	直接包装 MemoryService、KnowledgeCompiler、ExperienceService。默认可用。
L1 派生写入	analyze_qa、compile_knowledge	会写派生表和 output/knowledge；必须记录 extractor、scope、fingerprint。
L2 人工决策	review_document、resolve_merge	代表人的确认，不应由模型静默执行。必须由用户明确授权。
L3 来源变更	import_source、delete_source（未来）	改变事实层；要有预览、幂等键、影响分析和可撤销/补偿方案。

第一版 Agent 的结束条件必须是显式的：已找到足够证据、需要人工选择、达到步数/费用上限、工具连续失败、或证据不足。不要让“模型自己觉得可以继续”成为唯一控制条件。

3.3 建议的数据契约

```
AgentRequest  = {goal, project_id?, topic?, mode: 'read_only'|'draft'}
AgentAction   = {tool_name, arguments, reason, expected_evidence}
ToolResult    = {ok, data, evidence_ids, error?, truncated?}
AgentAnswer   = {summary, claims[], evidence_ids[], uncertainties[], next_action?}
Claim         = {text, provenance: 'source'|'ai_extension', evidence_ids[]}
```

把这些对象做成 JSON Schema，并在模型输出后再次验证。现有 OpenAI extractor 已经展示了这条思路：结构化输出 + 证据 ID 白名单校验。

4. 数据库审查与建议

4.1 现有 20 张表可以按五层理解

层	表	含义
元数据与导入	schema_meta, import_runs, projects	Schema 版本、来源导入、项目边界。
事实与定位	conversations, messages, sentence_segments, questions, question_answers	不可被摘要覆盖的来源事实和精确位置。
分析与缓存	analysis_cache, qa_units, qa_evidence, topic_aliases, qa_topics, merge_reviews	Q/A 派生、缓存、标签映射和人工合并任务。
知识编译	compiler_runs, topics, topic_evidence	一次编译如何产生主题，以及主题由哪些句子支持。
发布与审核	knowledge_documents, document_revisions, review_events	可读文件、不可覆盖修订、人工判断。

理解数据库时，不要按建表顺序死记。按“来源事实 -> 派生分析 -> 知识编译 -> 人工审核”的生命周期去画关系图，会更容易。

4.2 SQLite 继续使用的理由与迁移门槛

继续 SQLite（现在）	迁移 PostgreSQL（出现这些信号后）
个人本地使用；单进程或低并发；数据量可控；希望零运维；主要目标是理解领域模型和 Agent。	多个用户/工作区；并发写入；服务化部署；细粒度权限/RLS；队列 Worker；更强全文/向量检索；备份恢复与运维要求。
先补：迁移脚本、统一时间格式、FTS5 可行性、查询计划、备份恢复演练。	迁移前补：workspace/user/tenant_id；字段映射；JSONB/UUID/timestamptz；事务隔离；连接池；Alembic 等迁移工具；双写或一次性校验脚本。

4.3 为 Agent 新增的数据表（将来实现，不是现在改）

表	关键字段	原则
agent_runs	id, goal, scope_json, mode, status, model, prompt_version, budget_json, started_at, completed_at	一条用户目标对应一次运行；可中止、可重放、可比较。
agent_steps	id, run_id, step_no, action_type, tool_name, input_json, output_summary, status, latency_ms	追加式；不要覆盖失败步骤；大结果不要整段塞日志。
agent_step_evidence	step_id, sentence_id, relation	把 Agent 判断重新连回现有 sentence_segments。
agent_artifacts	id, run_id, kind, path/object_key, sha256, metadata_json	保存最终草稿、上下文包或导出，正文可放文件/对象存储。
agent_feedback	id, run_id, rating, decision, note, created_at	记录用户接受、驳回、修订；成为后续评测数据。

4.4 数据库具体改进顺序

1. 先建立正式迁移历史：每次 Schema 变更有版本、向上迁移、验证和备份说明；不要继续把所有演进塞进 initialize()。
2. 统一时间：SQLite 统一 UTC ISO 8601 TEXT，PostgreSQL 使用 timestamptz；不要同时混用 REAL 和 TEXT。
3. 补约束：role/status/action 等字段增加 CHECK；为必要外键和删除策略写测试。
4. 用 EXPLAIN QUERY PLAN 学索引：特别观察 LIKE '%query%' 为什么无法有效利用普通 B-tree。
5. 个人版先研究 SQLite FTS5；真正服务化后再评估 PostgreSQL 全文、pg_trgm 与 pgvector。向量用于召回，不作为最终证据。
6. 明确文件一致性：采用临时文件 + 原子替换，或让数据库记录 artifact 状态并支持补偿/重建。
7. 迁移 PostgreSQL 前做字段级对照表；infra/postgres/001_initial.sql 目前是规划草案，不是现有 SQLite 的等价迁移。

5. 新虚拟环境：建议现在就做

建议使用 Python 3.12。仓库要求 Python >= 3.11，本次审查也用 3.12.14 跑通 15 项测试。项目当前 dependencies=[]，因此虚拟环境很轻；它的价值主要是隔离解释器、未来依赖和学习数据。

5.1 Windows PowerShell 推荐命令

```
cd D:\mytools\GPT-SaaS
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e .
python -m unittest discover -s tests -v
```

如果 PowerShell 不允许激活，不必修改全局执行策略；可以直接调用 .\.venv\Scripts\python.exe 执行后续命令。

5.2 用独立学习数据库跑一遍

```
python -m ai_memory --db data/learning-sandbox.sqlite init
python -m ai_memory --db data/learning-sandbox.sqlite import examples/learning-project.json
python -m ai_memory --db data/learning-sandbox.sqlite analyze
python -m ai_memory --db data/learning-sandbox.sqlite compile --output output/learning-sandbox
python -m ai_memory --db data/learning-sandbox.sqlite topics
python -m ai_memory --db data/learning-sandbox.sqlite serve --port 8877
```

环境	数据库	用途
.venv + learning-sandbox.sqlite	可随时重建的示例数据	学习、断点、SQL 实验、Agent 原型。
.venv + dev.sqlite	较长期的脱敏开发数据	功能联调、评测集、迁移演练。
未来 production	PostgreSQL + 对象存储	只有部署、多用户和并发需求成立后再建立。

6. 代码阅读顺序：不要从 UI 开始

顺序	文件	带着什么问题读	读完的产出
1	tests/test_v01.py	一条导入怎样被判定成功？幂等和句子 offset 如何验证？	手画 V0.1 数据流。
2	connectors/base.py, generic.py	外部格式怎样变成 CanonicalMessage？	能手写一个最小导入 JSON。
3	service.py	稳定 ID、事务、问答关联、句子切分怎样工作？	跟踪一条消息落到哪些表。
4	database.py	20 张表哪些是事实，哪些是派生？外键怎样删除？	画 ER 图并写 5 条 JOIN。
5	tests/test_v02.py + compiler.py	指纹、增量、修订、审核怎样保证可重算？	解释一次 compile 的成功/跳过/失败。
6	extractors/base.py, heuristic.py	Protocol 如何让规则和 LLM 共用接口？	写一个不接 API 的关键词 extractor。
7	openai_responses.py	结构化输出、store=false、证据白名单在哪里？	能指出模型输出的信任边界。
8	tests/test_v021_v03.py + qa.py	缓存键、标签哈希、相似合并复核怎样分工？	能解释“哈希不是语义”。
9	experience.py, api.py, static/	数据怎样变成时间线、Context Package 和 UI？	能从 API 路由反查到 SQL。

每读一个模块都做三件事：写下输入/输出；画出读写了哪些表；找一个测试证明你的理解。不要只顺着代码逐行读。

7. 8 周渐进学习路线

按每周 6–10 小时设计。时间少时可以拉长，不要压缩验证环节。每一阶段都有一个看得见的产出，避免只看概念。

周	主题	具体知识点	动手产出 / 达标标准
第 1 周	运行与调用链	venv、editable install、模块/包、argparse、unittest、断点调试、Path。	在 learning-sandbox.sqlite 跑完整流程；能口述 import -> compile -> serve。
第 2 周	关系模型与 SQL	主键/外键、1:N/M:N、JOIN、GROUP BY、事务、rollback、WAL、索引、EXPLAIN。	画 20 表分层图；独立写出项目 -> 对话 -> 消息 -> 句子查询。
第 3 周	幂等与可追溯	SHA-256、UUID5、内容寻址、offset、provenance、cache key、revision。	解释重复导入为何跳过；人为改一条输入并预测哪些结果失效。
第 4 周	结构化 LLM	Protocol、dataclass、JSON Schema、提示版本、证据白名单、失败重试、隐私边界。	写一个 FakeExtractor 测试；拒绝未知 evidence_id。
第 5 周	单 Agent 与工具	AgentState、工具 Schema、read/write 分级、step limit、timeout、错误分类、终止条件。	只读 Agent 能回答主题问题并列出 sentence_id；不允许执行写工具。
第 6 周	运行账本与人工审批	append-only log、agent_runs/steps、审批门、artifact、审计、可重放。	一次运行可以从 DB 还原步骤；写工具必须显式批准。
第 7 周	检索与评测	关键词/FTS/向量候选、召回率、精确率、faithfulness、错误合并率、golden set。	建立 20–30 个脱敏问题金标；输出评测表而非主观感觉。
第 8 周	迁移与生产边界	Schema migration、PostgreSQL、JSONB/UUID/timestamptz、RLS、连接池、备份恢复、队列。	写 SQLite->PostgreSQL 字段映射和迁移触发清单；此时再决定是否迁。

7.1 每周固定复盘问题

- 我能指出这一步的权威数据源吗？
- 失败后会留下什么状态，能否安全重试？
- 一个结论能否回到具体 sentence_id？
- 模型生成内容与来源事实是否被清楚区分？
- 这个功能真的需要 Agent，还是确定性代码更可靠？
- 如果换模型、换数据库或换 UI，领域数据是否仍然成立？

8. 建议的实践项目：从读懂到真正拥有

练习 A：数据库侦探（第 1-2 周）

目标：不改代码，只用 SQLite 查询回答“某个主题来自哪些项目、对话、消息和句子”。

任务	验收
查询某个 topic 的全部证据链	输出 topic -> topic_evidence -> sentence_segments -> messages -> conversations -> projects。
查询一个 knowledge_document 的全部 revision	按 revision_no 排序并比较 content_sha256。
观察重复分析	第二次 analyze 命中 analysis_cache，解释 cache_key 组成。

练习 B：确定性“小 Agent”（第 3-4 周）

先不用模型：根据用户输入的 topic，依次调用 get_topic、timeline、context_package，组合结构化答案。这样你能先掌握工具契约和状态机。

练习 C：只读 LLM Agent（第 5 周）

模型只能从白名单选择工具，最多 6 步，最终每个 claim 必须引用 sentence_id。让 10 个固定问题重复跑，记录工具成功率、证据正确率和平均步骤数。

练习 D：需要批准的编译 Agent（第 6 周）

Agent 可以提出 analyze/compile 建议，但执行前展示 scope、extractor、预计写入位置和风险。review/merge 永远由人确认。

练习 E：检索评测（第 7 周）

为 20-30 个问题标出期望 sentence_id 和 topic；比较 LIKE、FTS5、可选向量召回的 Top-k。最终答案仍以真实证据为准。

9. 实现路线图：未来真正动代码时怎么排

阶段	建议变更	验收门槛
M0 基线	新增开发环境说明、固定测试命令、示例 DB 约定。	新环境 15/15 测试通过；数据不混用。
M1 工具层	为现有查询服务建立小型 ToolSpec/ToolRegistry，不改领域逻辑。	工具参数有 Schema；错误结构统一；只读工具无副作用。
M2 单 Agent	AgentState + 有限循环 + policy gate + evidence critic。	未知工具/证据被拒绝；步数/预算可控；终止原因明确。
M3 运行账本	增加 agent_runs/steps/evidence/artifacts/feedback 迁移。	失败也可审计；可重放；正文不泄漏到日志。
M4 评测	goldens、指标、回归脚本、模型/提示版本对比。	每次变更有量化报告；不能只看单次 Demo。
M5 检索	先 FTS5，再根据数据和指标评估向量。	Top-k 召回提升且证据定位不丢失。
M6 PostgreSQL	字段映射、迁移工具、多租户、RLS、对象存储、队列。	只有真实迁移信号出现；校验行数、哈希、外键和抽样结果。

9.1 不建议现在做的事

- 不要同时引入 LangChain/LangGraph、向量库、Redis、PostgreSQL、Docker 和多 Agent；你会失去对问题来源的判断。
- 不要让 Agent 获得任意 SQL、任意文件写入或任意 shell 工具；工具要窄、可验证、参数有 Schema。
- 不要把聊天上下文直接当长期记忆；长期记忆应进入明确的数据结构，并带来源与生命周期。
- 不要用向量相似度自动合并主题；现有 merge_reviews 的保守设计应继续保留。

- 不要在无鉴权情况下把本地 HTTP 服务绑定 0.0.0.0 或暴露公网。
- 不要把模型输出直接覆盖 source/message/sentence 事实层。

10. 风险清单与应对

风险	当前表现	建议控制
幻觉证据	OpenAI extractor 已拒绝未知 ID，但未来 Agent 多步后仍可能丢引用。	每个 Claim 独立校验；证据不足则降级为不确定。
无限循环/费用失控	目前没有 Agent loop，因此尚未发生。	最大步数、总 token/费用、单工具超时、连续失败熔断。
越权写入	现有 UI 有 review/refresh 写入口，但只适合本地。	工具分级、显式模式、人工批准、审计记录。
数据与派生文件不一致	DB 与 Markdown 分两阶段写。	原子文件替换、artifact 状态、可重建命令与一致性检查。
Schema 演进失控	initialize() 仅含极简升级。	迁移工具、版本化脚本、备份、回滚/前滚策略。
错误标签合并	当前近似标签会进 review，这是优点。	继续保守；评测 false merge；支持撤销与别名历史。
敏感数据泄漏	默认离线；外部调用需显式选择。	数据分级、脱敏、最小发送、日志不记正文、密钥只在环境变量。

11. 现在就可以执行的 10 项清单

1. ☐ 创建 .venv（Python 3.12），运行 15 项测试并保存基线结果。
2. ☐ 使用 learning-sandbox.sqlite，不把学习数据与未来真实数据混在一起。
3. ☐ 按 tests/test_v01.py -> service.py -> database.py 的顺序读第一遍。
4. ☐ 画出 20 张表的五层分组与关键外键，不追求漂亮，追求能解释。
5. ☐ 用 SQLite 客户端写 5 条 JOIN，完成“主题回到原句”的证据链。
6. ☐ 读 compiler.py，记录一次编译何时 skipped、compiled、failed。
7. ☐ 读 openai_responses.py，标出输入边界、输出 Schema、证据白名单和密钥边界。
8. ☐ 先写确定性的 ToolSpec 草图，不接模型、不改数据库。
9. ☐ 定义 AgentState、6 步上限、只读模式和 5 种终止原因。
10. ☐ 准备 20 个脱敏问题与期望证据，作为后续 Agent 金标集。

> 完成前 5 项，你就已经不再是“对仓库不熟”；完成前 8 项，你已经具备实现最小 Agent 的能力；完成全部 10 项，才值得进入模型选择、向量检索与 PostgreSQL 迁移。

附录 A：审查证据与定位

证据	仓库位置
版本、Python 要求、无第三方运行依赖	pyproject.toml：1–17
默认 SQLite 路径与 CLI 命令	ai_memory/cli.py：17–98
20 张实际表、WAL、外键与索引	ai_memory/database.py：10–236
数据库初始化与事务提交/回滚	ai_memory/database.py：240–270
导入、哈希、稳定 ID、句子和 Q/A 关系	ai_memory/service.py：16–153
Q/A 分析、缓存和标签复核	ai_memory/qa.py：43–335
知识编译、来源指纹和修订	ai_memory/compiler.py：33–486
Extractor Protocol	ai_memory/extractors/base.py：1–34
离线默认规则提取器	ai_memory/extractors/heuristic.py：17–79
Responses API、store=false、未知证据拒绝	ai_memory/extractors/openai_responses.py：39–159
本地 HTTP 服务	ai_memory/api.py：16–153；ai_memory/cli.py：88
PostgreSQL 规划草案	infra/postgres/001_initial.sql：1–94
架构决策：先可追溯内核	docs/adr/0001-architecture-baseline.md
15 项测试	tests/test_v01.py，test_v02.py，test_v021_v03.py

本次测试命令：Python 3.12.14，python -m unittest discover -s tests -v；结果 15/15 通过，用时约 1.8 秒。检查时 data/ 目录不存在，因此默认数据库尚未初始化。

附录 B：迁移 PostgreSQL 的决策表

问题	若答案为“是”	若答案为“否”
是否已有两个以上真实用户并发写入？	开始迁移评估。	继续 SQLite。
是否需要按 workspace/user 强制数据隔离？	设计 tenant_id、RLS 与鉴权。	先保持本地单用户。
是否需要 Worker/队列并行处理？	评估 PostgreSQL + 队列。	单进程更简单。
LIKE/FTS5 是否已无法满足检索指标？	用金标评估 PostgreSQL 全文/向量。	不要为技术潮流迁移。
是否具备备份、恢复、监控和迁移测试能力？	可以进入生产迁移。	先补运维基础。

附录 C：核心术语速查

术语	在本项目中的意思
事实层	项目、对话、消息、句子及其来源定位；不应被摘要覆盖。
派生层	Q/A、主题、摘要、标签、知识文件；可以重算和修订。
Provenance	某条结论从哪里来，如何回到原始句子。
幂等	同样输入重复执行，不产生重复事实或意外副作用。
指纹	对输入与配置做哈希，用于判断是否需要重算。
Agent 工具	Agent 可以选择调用、参数受 Schema 限制的窄函数，不等于任意代码执行。
运行账本	一次 Agent 运行中每一步决策、工具、结果、错误与成本的可审计记录。
RAG	先检索候选证据，再让模型基于证据生成；检索结果仍需权限与引用校验。
Human-in-the-loop	模型提出建议，人对高风险写入、合并或发布做最终确认。